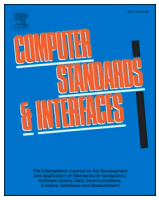
Computer Standards & Interfaces 89 (2024) 103832



Contents lists available at ScienceDirect

Computer Standards & Interfaces

journal homepage: www.elsevier.com/locate/csi





A critical analysis of Zero Trust Architecture (ZTA)

Eduardo B. Fernandez [a,*], Andrei Brazhuk [b]

[a] Florida Atlantic University, Boca Raton, FL, USA
[b] Yanka Kupala State University of Grodno, Grodno, Belarus

ARTICLE INFO

Keywords:
Zero trust
Zero trust architecture
Secure system architecture
Secure system design
Security patterns
Reference architectures

ABSTRACT

Zero Trust (ZT) has become a very hot approach for building secure systems, promoted by industry and government as a new way to produce systems with a high degree of security. ZT is based on not trusting any request for accessing resources. Because of the possibility of increasing the security of enterprise systems there has been a large amount of publication on different aspects of this strategy. It is then important to evaluate if its claims are true. We have used security patterns to design and evaluate security architectures and we apply here this method to analyze the expectations of this strategy. We relate the ideas behind ZT to the accumulated knowledge of security and attempt to answer some questions about the value and possibilities of this technology. In general, industry publications are vague about the technical aspects of these systems, ignore past security knowledge, and there are few reports describing actual experience building and using ZT architectures. Is Zero Trust Architecture (ZTA) the ideal architecture to build secure systems? To obtain a deeper understanding of this architecture, we analyze its pattern structure and provide a sketch of its reference architecture built as an aggregation of security patterns. As any system architecture, regardless of the way it has been constructed, represents a system, we also consider its threats. Finally, we provide directions for research on this area.

1. Introduction

Zero Trust Architecture (ZTA) or just Zero Trust (ZT), is being touted as a new concept that will significantly increase the security of organizations. A driving force behind the need for ZTA is the complexity and heterogeneity of modern IT systems. A company may own several internal networks, and be connected to a variety of external networks, such as remote offices and partner networks. Enterprise applications may be hosted locally or be dynamically run from clouds and storage services. Also, enterprises are integrating their IT systems with the Internet of Things (IoT) and with cyber physical systems; different operational approaches may be used, such as Bring Your Own Device (BYOD), and Work From Home (WFH). The effect of the pandemic has made systems very fragmented where users need fast access to a large variety of applications and resources deployed across an increasingly distributed infrastructure. Access to systems comes now from a more diverse type of devices and from more locations. Scattered and fragmented systems have a large attack surface, and combinations of office and home systems may aggregate systems with incompatible security models, which makes system management difficult. At the same time, attacks have become more sophisticated and more frequent.

ZT has been introduced as a strategy or philosophy to build secure systems, not as a specific architecture, which makes things rather unclear for potential implementors. Even worse, there are very few reports of actual implementations; those which exist are mostly descriptive and not very explicit about technical details. Several IT vendors claim to have ways to implement a ZTA, but they provide few details about how to do this; white papers from these companies describe their approaches, but they only provide high-level overviews or lists of specifications.

A fundamental concept of ZT is the application of a set of principles, taken from the classical work of Saltzer and Schroeder [1] and subsequent works [2], that may include closed system, least privilege, complete mediation, defense in depth, layered defenses, and others, to which ZT adds “trust no one”. The problem with principles is that they are abstract concepts, there are many ways to implement them, and they do not provide guidance on how to apply them. Also, assurance that these principles are enforced requires consideration of all the architectural levels of the system. Different products may adopt different sets of principles, and it is not clear what is an optimal set of principles. Because of the difficulty in direct application of principles, proposals for ZTAs adopt specific mechanisms, but highly secure systems cannot be built by just adding security mechanisms to a system; we need a systematic

* Corresponding author.
E-mail address: fernande@fau.edu (E.B. Fernandez).

https://doi.org/10.1016/j.csi.2024.103832
Received 4 July 2022; Received in revised form 7 December 2023; Accepted 8 January 2024
Available online 9 January 2024
0920-5489/© 2024 Elsevier B.V. All rights reserved.

secure development methodology where threats are identified, and countermeasures are added to handle them [3]; that process seems like the best way to build demonstrably trustworthy systems [4]; using security defenses which are not needed may result in intolerable overhead, higher cost, and even lower security.

An effective method to understand, design, and evaluate security architectures is the use of patterns [5,6]. A pattern is an encapsulated solution to a recurrent software or system problem. Security patterns describe how to control threats or correct a vulnerability [7]. Using security patterns, it is possible to analyze an existing type of system to evaluate its security properties. We analyze here the fundamental security aspects of ZTAs using security patterns. Patterns can show that a ZTA is a combination of well-known security concepts that usually have been ignored by industry and that there are still several unclear points that need more study before we can get to successful implementations that can deliver highly secure systems. This does not mean ZTA is not a valuable strategy, but it is important to separate its valuable aspects from the hype and study how to structure it to make it highly secure. As it has been demonstrated in recent ransomware attacks, many institutions have not invested any significant resources to protect their assets. This is especially true when the data belongs to their customers, e.g., Equifax [8], but they even neglect protecting their own data, e.g., Sony [9]. If ZTA can help bring more security awareness to institutions, it will be a positive force for change. In fact, although the ideas behind ZT are not new, there are many new tools and ideas that can make its realization more convenient.

We analyze here some of the proposed ZTAs, consider their core features and implementations, and evaluate them with respect to the classical security knowledge. We use security patterns as artifacts of an abstract framework to evaluate these architectures and define possible ways toward correct implementations.

Our contributions are:

- A description of the evolution of the main architectural proposals for ZTA from industry, highlighting their main features and the principles they use, as well as some common implementation aspects. We try to understand what practitioners understand by ZT and to show that there are diverse interpretations of it.
- A discussion of the relation of ZTA to classical security theory. We show that the basic concepts behind ZT were already there.
- An enumeration of architectural security mechanisms in ZTA in the form of security patterns, to expose its core abstractions. We discover patterns in the proposed ZTAs.
- A security reference architecture, which is preliminary but more precise than existing ones. This points to future work.
- Preliminary answers for important practical questions about ZTA, which we believe should be answered before this technology is widely applied; these are effectively directions for research, including:
  - Q1. Does ZT apply many unnecessary checks?
  - Q2. Does ZT introduce an intolerable overhead?
  - Q3. Is ZT implementable for most institutions?
  - Q4. Does ZT control more, fewer, or similar number of threats as compared with other approaches?
  - Q5. Does ZT produce a huge number of policies?

This work is organized as: Section 2 provides background on security principles, security patterns, and reference architectures. Section 3 presents the main ideas of industry proposals for ZTA. Section 4 discusses relevant security theory and its relation to ZTA, it also places ZTA in perspective by using patterns and defining a preliminary security reference architecture. Section 5 answers Q1 to Q4. Section 6 provides a list of directions for research. Section 7 presents related work, and we end with some conclusions in Section 8. An appendix defines the abbreviations used in this article.

2. Background

ZTA applies several security principles to guide its architecture as well as its implementation. We enumerate the principles we have identified in a variety of articles. Since we are using security patterns and security reference architectures to analyze and understand this type of architecture, we provide their definitions.

2.1. Principles

The design of software-based systems requires to apply some principles to help us build systems that have appropriate levels of security, availability, reliability, performance, safety, maintainability, or usability. These are the non-functional requirements or quality attributes of the system. Different applications have different quality requirements; we consider here applications that require a high level of security. Saltzer and Schroeder in a classic paper [1] proposed a set of fundamental principles that should be applied to build secure systems; they include:

- *Closed system*. Anything not explicitly allowed is forbidden. This principle is applied to the rights that control resource access during execution, and to the rights given to users. This is a fundamental principle, applied in all ZT systems.
- *Least privilege (need to know)*. Users or roles should be given only the specific rights they need to perform their jobs. This is another principle applied in most ZT systems.
- *Complete mediation*. Every request for access to resources must be validated. This is another fundamental principle for ZT systems.
- *Open design*. Exposing the security design allows to verify that the design is sound. Only specific data, necessary to use the system should be secret, e.g., passwords and keys.
- *Economy of mechanism*. The security mechanisms should be as simple and small as possible. This makes comprehension, testing, and analysis easier.
- *Minimal trust*. The parts of the system that must be trusted should be minimal. This does not refer to users or devices, as in ZT, but to the units of the system that must be trusted by the rest of the system.
- *Separation of privilege*. Critical actions need two independent mechanisms (users, components, systems) to agree before being performed.
- *Least common mechanism*. The amount of mechanism common to two or more subsystems should be minimized.
- *Safe defaults*. Failures (crashes) should close everything for access.

Later, other principles were proposed:

- *Highest level*. Security constraints must be defined at the highest layer (application), where their semantics are clear, and propagated to the lower levels, which enforce them. We consider this is a fundamental principle often neglected in industry.
- *Holistic security*. All the layers of the architecture must be secure, and we must apply security in all development stages. That is, the secure lifecycle must be two-dimensional, considering time and space [7, 10].
- *Ease of use or transparency*. Users must accept the security system, or they will not use it. Security functions must be transparent or at least easy to use.
- *Information hiding and encapsulation*. Implementation details should not be exposed, only a clear, functional interface should be shown. This principle comes from [11] and points toward object-oriented design realizations.
- *Defense in depth*. We should have more than one line of defense.
- *Compartmentalization*. We should isolate units that do independent work at execution time.

- *Submarine principle.* We can accept to lose some units, but we must preserve essential units. This principle is based on the fact that not all units of a system have the same sensitivity or importance, and it is also a survivability principle.
- *Compromise recording.* In some cases, mechanisms that reliably record that a compromise of information has occurred can be used in place of more elaborate mechanisms that completely prevent loss.
- We can also define principles for specific systems, such as "no application can execute with root privilege" [12].

Principles are very important, but they are just higher-level guidelines that do not indicate how or where to apply them. This means that inexperienced designers have a hard time trying to apply them. In fact, a motivation behind security patterns is to define constructive guidelines that embody principles. We have attempted to build patterns to illustrate the use of some principles, although we only produced one (least privilege principle) [13].

Specific ZTAs select different sets of principles but they all seem to include at least closed systems, least privilege, and complete mediation.

2.2. Security patterns and reference architectures

A pattern describes a solution to a recurrent problem that occurs in a given context [5,6]. This solution is described using a template composed of a set of structured sections, which makes easier and more effective the use of patterns. A context section describes where the pattern can be used, e.g., distributed systems, and preconditions for its use, such as "the requestor must first be authenticated". A problem section describes the problem solved by the pattern and the forces that constrain its solution, e.g., "the solution must have a low overhead". The solution is first described in words and maybe with block diagrams; it is then made more precise by using UML diagrams, usually class and sequence diagrams, and possibly also activity and state diagrams. In critical applications, a pattern diagram may be complemented with formal expressions. Pattern solutions are suggestions, not software components; they must be instantiated and tailored, their main purpose is to provide ideas to inexperienced designers. An implementation section provides guidance on the use of the pattern, indicating what steps are needed and their order, possible standards or regulations to follow, and examples of industry implementations or code samples. A section on "Known uses" lists at least three or more actual systems where this solution has previously been used, i.e., patterns are abstractions of good practices. A section on consequences indicates what is the effect of the pattern when used to solve a problem and how well the forces have been satisfied by the solution, including the advantages and disadvantages of using the pattern. A "Related patterns" section indicates other patterns that complement this pattern or that provide alternative solutions; for example, a procedure to perform a task could have a more complex but more precise solution. Variants of the pattern, corresponding to specific contexts, may also be shown.

A pattern embodies the knowledge and experience of designers and can be reused in new applications; a well-built pattern implicitly applies good design principles. Patterns for common problems may be used to communicate solutions to other designers, as building blocks for complex architectures, and to evaluate and reengineer existing systems. In particular, design patterns solve software construction problems, such as making a solution more flexible. Security pattern templates include all the sections of design patterns plus a section on threats that lists the generic threats that can be handled by the pattern and their corresponding defenses. Over two hundred different security patterns have been published that touch all aspects of the design of secure computer systems, such as Access Control, Network Security, Web Services Security, etc. [7]. More are under development for relatively new fields, such as IoT [14], and Machine Learning [15].

A Reference Architecture (RA) is a generic and abstract software architecture, based on one or more domains, with no implementation aspects [16]. A domain is an area of knowledge and normally defines only functional aspects, although it can also define conceptual security or safety aspects. Domains and reference architectures can be defined using UML class diagrams, ontologies [17], or combinations of them. An RA can be built using patterns and it can be considered as a kind of pattern for whole architectures; from an RA we can derive concrete software architectures by adding implementation aspects. RAs are useful to enumerate threats by indicating the places in the system where there may be possible vulnerabilities that can be exploited by threats. If we add security patterns to control the identified threats in an RA, we obtain a Security Reference Architecture (SRA) [18].

Security patterns and SRAs are created and validated by induction (comparing them to known solution instances), and deduction (by enumerating their threats and verifying that the pattern or SRA covers these threats).

3. ZTAs in industry

As indicated earlier, since ZT is mostly a strategy there are many ways to interpret it. We show here some of the attempts to define a ZTA in industry. For each proposal, we identify the principles it applies (explicitly or implicitly) and some features of its architecture. Most vendors give few implementation details of their systems, perhaps since they consider these aspects to be proprietary or they have done few actual implementations.

A *network-driven architecture of zero trust* was proposed by Forrester [19]. It used most of the Saltzer's principles (closed system, fail-safe defaults, least privilege, separation of privilege, network isolation, complete mediation, compromise recording, continuous protection, and defense-in-depth). As the name indicates, it was a network-centric view, the main idea was to bring security to end client connections. This architecture emphasized a network segmentation approach (a realization of the compartmentalization principle, see Section 4), and centralized network management.

Zero Trust Network Access (ZTNA) is a variant of the Forrester idea, with several new features. ZTNA checks the credentials of a (user, device) tuple as subject to verify that it has permission to access an application; it then creates a secure connection (encrypted tunnel) and logs and monitors the connection. ZTNA should be able to be seamlessly integrated with mechanisms like SD-WAN, SASE, and cloud-based security to ensure consistent protection; it should also work with any transport method or network, including broadband, LTE, and 5 G. By creating a unified, security-driven networking strategy, institutions could automatically adapt to changes and expansions without affecting access or security [20,21].

Zero Trust Extended (ZTX) extends the zero trust concepts to data, network, people, workloads, and devices. Forrester has built a framework to implement this model, including strategy, capability, technology, and features; however, their report [22] gives no details of it.

Google implemented the BeyondCorp project, where they combined existing artifacts and focused on access control and device accounting. They seem to follow the standard principles, but with emphasis on the application (*application-style zero trust architecture*) [23]. An objective was to remove the requirement for a privileged perimeter and move corporate applications to the internet, allowing remote enterprise entities to interact with remote applications. Google built their approach from well-known items like access proxy, Single Sign-On (SSO), and Radius authentication (all these are patterns in [7]). An access control engine maintains all pipelines between users and enterprise resources, based on trust inference, taking in consideration the data in a user database and in a device database. Devices and users are assigned levels of trust, which are evaluated dynamically. The level of trust can then be used by the Access Control Engine (described below) as part of its decision process for requests. This work, and the architecture proposed by Akamai [24], emphasized application control instead of network control; this idea applies the principle that the sensitivity of the information

depends on its semantics, which is not known at the network level.

NIST summarized ZTA work up to early 2020 [25]; in their model, authentication and authorization follow dynamic policies and access to a resource (application, service, or data) for a subject (user, device, or application) is only given during a communication session (temporarily). This model assumes that a subject's credentials are not the only authorization factors, these factors also can include network location, software and security states, device used, and behavioral aspects.

Fig. 1 shows the logical components of NIST's model. The Policy Enforcement Point (PEP) is responsible for interaction with subjects and for forwarding their access requests of resources to the Policy Decision Point (PDP). PDP is used to manage access policies and authentication decisions. PDP consists of Policy Engine and Policy Administrator. The Policy Engine makes the final access decision, while the Policy Administrator controls the communication from subject to resource. The Data Access Policies (authentication and authorization information) are the major determinants of access. NIST applies the least privilege principle in assigning rights. A set of extra security features are also applied, including Continuous Diagnostics and Mitigation (CDM), industry compliance, threat intelligence, activity logs, Public Key Infrastructure (PKI), ID management (used for authentication and device identification), and Security Information and Event Management (SIEM). Control activities are assigned to a control plane; data is communicated in a data plane. This model also considers a *trust algorithm*, used by the policy engine to make its decision. Trust is calculated using as inputs the policy database with information about subjects, subject attributes and roles, historical subject behavior patterns, threat intelligence sources, and other metadata sources. This is a comprehensive model, but it does not indicate when the specific inputs are considered; for example, Industry Compliance are just policies, defined by the system administrator, SIEM is a type of Cyber Threat Intelligence (CTI), but their use is not detailed. The use of a control and data planes is not appropriate in a conceptual model, those are really implementation aspects. We use this model in Section 5 as a basis to develop a Security Reference Architecture.

Several extensions to the NIST architecture exist. Gartner proposed a reference framework developed by Qi Anxin [26], based on several fundamental properties like identity-based schema, continuous trust evaluation, resource secure access, and adaptive access control. They added their own system of principles (comprehensive identity, application-level control, closed loop security, business aggregation, multi-scenario coverage, and component high interactivity). The Qi Anxin group defined an authorization model where access objects are any system resources and subjects are dynamic combinations of digital entities: (user, application, device). Their report does not mention access types. They adjust permissions dynamically according to trust levels, applying a continuous trust evaluation; all application traffic is encrypted using TLS as communication protocol.

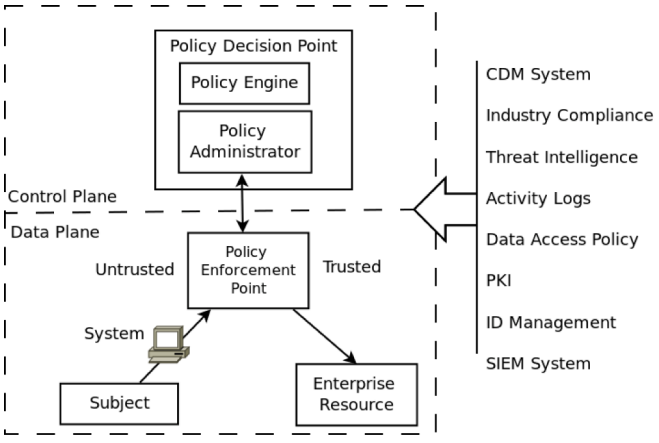


Fig. 1. NIST's conceptual ZTA model.

In ZTA, identities are fundamental for authentication. Network-driven architecture said little about user identities. Its extension, ZTNA, as well as the application-style architecture operate with two kinds of identities: user and device. ZTX has a wider view of this aspect, it also mentions people identities. The NIST architecture and Ref. [26] enumerate several kinds of subjects: users, services, applications, and devices. Qi Anxin [26] emphasized a concept of network agents (authorized subjects) as composite identities.

Publications about specific ZTA implementations are scarce. Microsoft implementation, a work in progress, allows full or no access to a system, there is no concept of access type [27]. Authentication is based on location, device, user/application, and risk, using their Active Directory. They use specialized segments, including purpose-built segments for the various IoT devices and scenarios used throughout the organization. Users and devices are segmented across purpose-built networks, migrating all Microsoft employees to use the internet as the default network, and automatically routing them to appropriate network segments. Google's BeyondCorp has a device database that keeps track of the devices managed by the enterprise [23]; only managed devices can access corporate applications. Devices are identified by their certificates, and they keep track of changes made to the devices throughout their lifecycle. Some research works are dedicated to implementations of zero trust test beds [28,29]. The National Cybersecurity Center of Excellence (NCCoE) of NIST is working on a ZT implementation document [30]; in this project, they use commercially available technology to build interoperable, open, standards-based implementations that align to the concepts and principles in the conceptual NIST model. An industry publication [31] has similar objectives.

4. ZTA, patterns, and security theory

We consider it important to relate ZTA to security research, expressed by the security patterns that realize basic mechanisms as well as describe threats. We first look at patterns for authorization and identity, network aspects, then at all the security patterns that may be needed in complete architectures and the ways to structure them, and finally at threat analysis and governance.

4.1. Authorization and identity models

We mentioned in Section 2.1 the origin of the principles used in ZTAs. Several of the security patterns included in ZTAs come from work on principles, authorization models, application firewalls, and XACML (eXtensible Access Control Markup Language) [32]. A problem is that NIST and vendor publications show only block diagrams, which are not precise enough to precisely define conceptual aspects. We describe below the relevant patterns, showing UML models for the most important ones.

Access control requires to apply authentication followed by authorization. Multifactor authentication is the accepted authentication method, and depends on identification, discussed below; industrial proposals normally use it. Their use of authorization is not so clear. The most basic authorization model is the access matrix, that is described by the tuple (s,o,t,p,f), where s is a subject (active entity), o is a protection object, t is an access type (read, write, hire, promote,…), p is a predicate that defines an access condition, and f is a copy flag that allows to copy a right that a subject has [33]. The predicate allows defining content-based authorization, fundamental for database systems. Most systems use the Role-Based Access Control (RBAC), where rights are assigned to roles, not to individuals; that is, the only accepted subject is a role [7,33]. More advanced models include the Attribute-Based Access Control (ABAC), where subjects and objects can be described by sets of attributes that may include location, rank, role, context, etc. [34]. ABAC provides a finer granularity of access than RBAC. The XACML model, described below, considers how individual authorizations can be combined to define very specific access conditions. The UCON model

expands the scope of authorization and considers governance aspects such as obligations and access conditions [35]. Authorization models define the unit of controlled access, but they need to be complemented with an enforcement mechanism; this is the concept of *reference monitor*. A reference monitor intercepts attempts to access resources, consults its authorization information (access evaluation) and decides if access is total, partial, or denied. Decisions can have more outcomes if we use a *Reified Reference Monitor* pattern [7]. A complete authentication/authorization (access control) system combines models for these two patterns, represented by the Identity Base and the Policy Base), as shown below for the application firewall, which controls access to application services. As can be seen from Fig. 2, the *Application Firewall* pattern [7, 36], includes a Policy Authorization Point (PAP) that includes Authorization rules (Policy Base), and an Identity Base, used for authentication. A Policy Enforcement Point (PEP) receives requests for access. A Content Inspector verifies that requests do not include suspicious signatures. Requests for access to services from clients are sent to a Policy Enforcement Point (PEP) that invokes the PAP and the Content Inspector (checks for well-formed requests) to decide access. The proposed ZTAs clearly use these ideas.

XACML, now in version 3.0, is a general access control language, intended to define and enforce authorization rules. It defines a very precise and rather complex way to specify access conditions. Its authorization rules can be described by the *XACML Authorization pattern*, while its enforcement is defined by the *XACML Access Control Evaluation pattern* [7,37].

The XACML Authorization pattern defines Policies, which are structured according to a *Composite pattern* [6], where a Policy Set is the composite element. When evaluating a PolicySet, Policies are combined according to the PolicySet's Policy Combining Algorithm, indicating how a specific policy is associated with other policies. Several policies may map to a Rule (can be realized by it). Several rules map to a Target that contains subjects, resources, environment, and actions (access types). A Policy Administration Point (PAP) is intended for security administrators to manage policies and rules (this is equivalent to the policy part of the PAP of the application firewall). This model allows defining policies much more complex than RBAC or ABAC.

The XACML Access Evaluation pattern (see Fig. 3) separates the Policy Decision Point (PDP) from the decision itself (called Access Response). A Policy Information Point (PIP) adds information to decide access. A Context Handler converts requests and decisions to external formats and takes information from the PIP.

XACML turned out to be too complex for use in industry and currently is not much used [38]. While ZTA has taken ideas from it, none of the commercial versions appears to use it. In particular, the PDP of the NIST model merges the PDP, PAP, and PIP of XACML and uses relatively simple policies, such as RBAC or ABAC. However, it may be necessary in systems with complex authorization requirements.

When defining authorization rules all proposals agree on using the least privilege policy [1], so that there is only the least amount of data and resources available to a subject. The least privilege policy can be applied with dynamic trust-based access control (see Section 4) to provide granular visibility of data.

As indicated above, identity is a fundamental technology when dealing with heterogeneous systems composed of many interacting devices and many users. Three basic aspects of identity can be described by patterns [7,39]: The *Identity Provider pattern* describes the administration of a security domain where identities are created and managed; the *Circle of Trust* structures a federation of service providers that trust each other; the *Identity Federation pattern* allows the formation of a dynamically created identity within an identity federation consisting of several service providers. Fig. 4 shows the Circle of Trust pattern, where a Circle of Trust has as members several Service Providers. Service providers are associated by trust where a service provider trusts one or more service providers, a relationship is described by an OCL invariant assertion which prescribes that the trustees of a service provider must be members of the circle of trust of the trustor.

Security logging, an important security mechanism, described as a

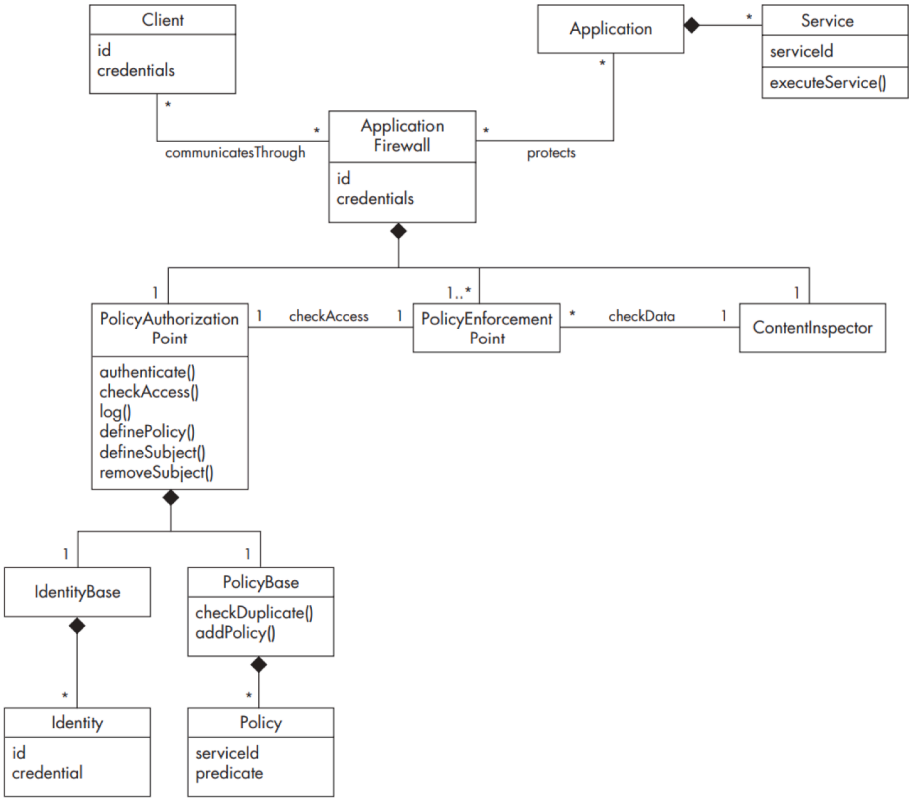


Fig. 2. Class diagram of the Application Firewall pattern (from [7]).

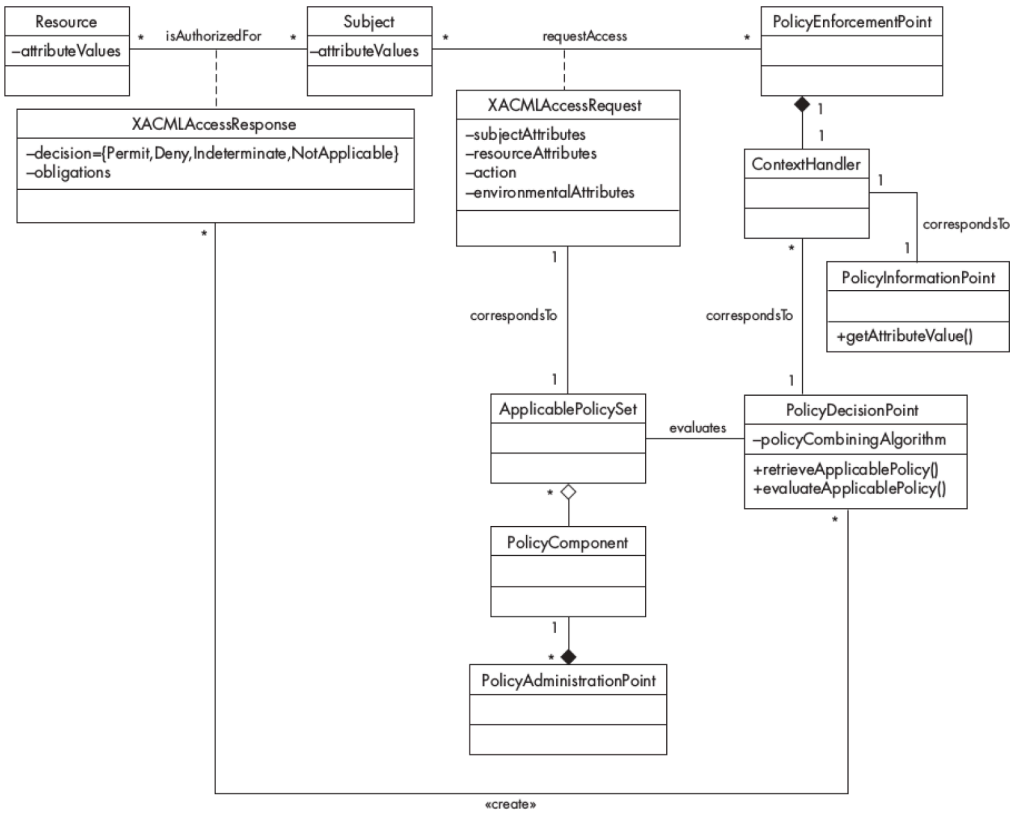


Fig. 3. XACML Access Control Evaluation pattern (from [37]).

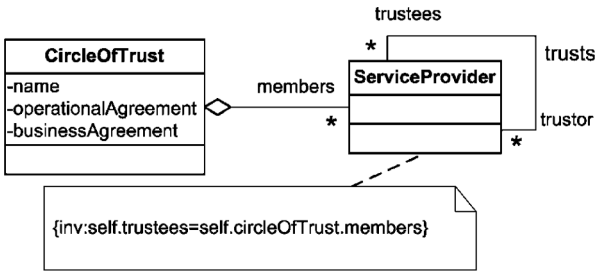


Fig. 4. Circle of Trust pattern (from [39]).

pattern in [7], is also mentioned in the descriptions of ZTAs. There is mention of activity logs in the NIST and Microsoft's [27] models. There is no explicit mention of audit activities in any system, although they may be included in SIEM software. Current systems must be accessible through mobile phones and various devices; CISA has discussed reference architectures for mobile ZT systems [40].

4.2. Network models

Network segmentation is a common pattern used to separate sections with different protection requirements [41]. Microsegmentation, a variant of segmentation, can be used to create protected segments of arbitrary sizes for resources behind gateways [42]. Means to implement micro-segmentation include firewalls, activity monitoring, access control, content control, intrusion detection and prevention, and cryptography. Forrester's architecture for a network driven ZTA is based on microsegmentation. An important item of this their approach is a segmentation gateway (SG) able to create Microcore and Perimeter (MCAP) segments. Also, the architecture includes a Data Acquisition Network (DAN) MCAP, to which all network related events and mirrored traffic can be forwarded, where security information management and network analysis centrally capture, analyze, and log all the data. This architecture aims to move information security as close as possible to end network connections (servers, users, applications, wireless networks). As a drawback, remote enterprise entities (like users working from home and enterprise public cloud applications) are out of the scope of this architecture.

Most network driven architectures use microsegmentation to define segments of different sensitivity, that can be separated using Secure Gateways (SG); for example, financial and personnel applications would have their own microsegments. An SG can be realized with several types of network devices, like UTMs (Unified Threat Management) and NGFWs (Next Generation Firewall), sold by various vendors. As an example, a network virtualization platform VMWare's NSX [43] allows deploying different security policies for specific microsegments using gateways (including automatic policy creation [44,29]).

NIST's ZTA mentions the use of Software Defined Perimeter (SDP) [45,46,47]. An SDP controls access to resources based on identity, usually applying a least privilege policy. The architectural ideas of SDP include the participation of clients in the control plane (client-to-gateway model) that can be implemented as a Single Packet Authorization (SPA), a variant of port knocking, and securing communications (TLS) [48]. Another technology, Software Defined Network (SDN) architecture, has brought new opportunities for network design [49]; the control plane is implemented in software in servers separate from the network equipment and the data plane is implemented with standard network equipment. In particular, [50,51,52] have proposed to combine SDN and SDP to provide a scalable and manageable security solution that can be used in ZTA.

Another important implementation issue is the use of cryptography in the system. Some proposals talk of encrypting everything and protecting all assets equally; several companies produce hardware devices to decrease the overhead of encryption, but that approach increases costs and reduces flexibility, there is no way to enforce different access types for example. Encryption is an effective way to protect network

traffic, but the appropriate use of authorization is a more flexible approach to protect fine units of data. In some ZTAs, there is talk of PKI but not of symmetric encryption, needed for use in database and communications encryption; symmetric encryption may be implied in the use of protocols such as TLS, but database encryption is not considered. The NIST and other models assume the use of PKI but do not indicate where to apply specific cryptographic controls. Patterns for symmetric and asymmetric cryptography as well as for digital signatures can be found in [7]. Performance studies of the use of cryptography in ZT networks are lacking.

4.3. Patterns and SRA in ZTA

From the analysis of proposed ZTAs and related models we can discover several patterns that can be used to build a reference architecture for ZT. Fig. 5 shows a pattern diagram with some of the patterns that can be recognized in existing ZTAs or potentially be used in new ZTAs. A pattern diagram shows relationships between patterns [5]. Patterns are shown as rounded rectangles; each directed arc from pattern A to pattern B indicates the contribution of pattern A to the pattern it points; for example, XACML provides a way to define both Authorization rules and Access Control Evaluation for the Authorizer. We have added UML associations to the pattern diagram model of [7]; e.g., SSO and Multifactor are special types of Authenticators.

Security Layers is an important pattern to structure secure systems [7], useful to separate concerns and hide critical system functions. The standard architectural layers usually include application, operating system/databases, middleware, networking, and hardware, sometimes with further subdivisions. ZTA work only considers control and data layers. Two layers appear insufficient to characterize the distribution of threats [53].

The *Controlled Execution Domain (Sandbox)* pattern describes how to define a secure execution environment for processes, indicating explicitly all the resources a process can use during its execution, as well as the type of access for these resources [7]. Some ZTA proposals mention this pattern.

Most systems interact with clouds. To access applications in the cloud, users are controlled by a *Cloud Access Security Broker* (CASB) [54]. The cloud broker sends requests to an authentication service. After a successful authentication, the cloud broker evaluates the corresponding policy and decides access to the application.

Except for UCON and Multifactor Authentication, they can all be found in [7]. Note the importance of the Authorizer pattern that defines rights for processes in execution in the Sandbox pattern, for controlling access to cloud applications in the CASB pattern, and for defining the rights enforced by gateways in the Segmentation pattern. In any of these cases, the Authorizer can use RBAC, ABAC, or other ways to define rights (see 4.4). The UCON pattern (not yet written) defines usage aspects of the authorization rules. XACML does not define a specific authorization model, it is just a notation to represent authorization rules, their combinations, and their enforcement; it represents here the fact that it is possible to use any authorization model. The Security Logger/Auditor pattern records the actual authorizations issued by the rule evaluation.

For simplicity, we left out in Fig. 5 patterns for Security Layers and for network aspects. A RA for ZT should incorporate all these patterns. Note that we have shown abstract patterns, they need to be adapted to consider the context of the specific type of applications; for example, financial applications require a higher level of security than game-playing applications, and therefore authentication and authorization need to be stricter for them. These patterns can be used to build ZTAs incrementally and also to evaluate the degree of security reached by a specific architecture; for example, if a system does not have a CASB it has no convenient way to use a cloud securely.

It is now possible to sketch a RA. A complete RA is a significant amount of work, and we only show a preliminary SRA based on the NIST ZTA of Fig. 1. Fig. 6 is a UML class diagram where classes have the same

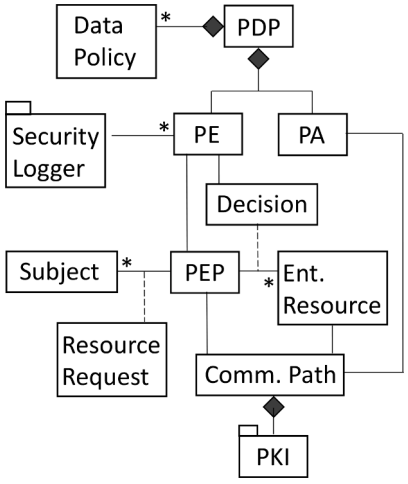


Fig. 6. A preliminary SRA for ZTA.
PA=Policy Administrator, PD=Policy Decision Point, PE=Policy Engine, PEP=Policy Enforcement Point.

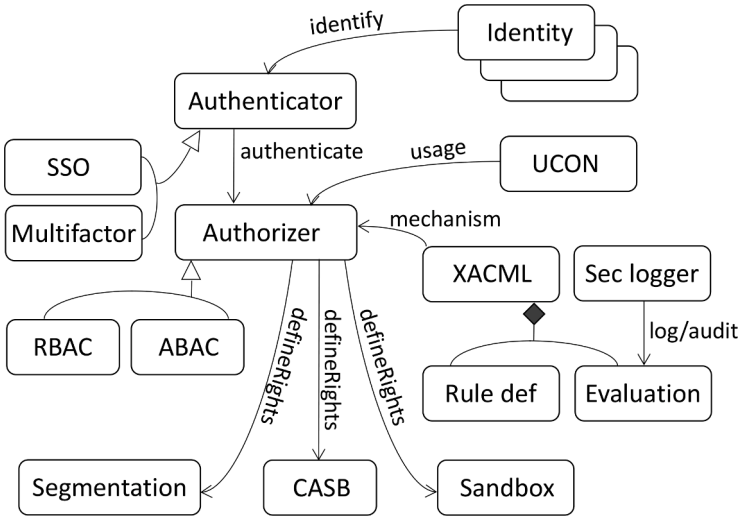


Fig. 5. Patterns used in ZTA.

meaning as the blocks in Fig. 1; however, this model is more precise than the block diagrams shown in all the publications about ZT that we have found. We have left out some of the inputs in that model because they may not be directly used to decide access because of their potentially high overhead. This is again a conceptual model using abstract patterns and system interfaces are not shown. Multiplicities of 1 and association names are not shown for simplicity; for example, there is only one PE and one PEP. Security Logger and PKI (Asymmetric Encryption) are patterns that can be found in [7]. Classes Subject, Resource Request, PEP, Enterprise Resource, Data Policy, and Decision are parts of the Reified Reference Monitor pattern shown in [7], with some name changes. Data Policy represents authorization rules to access data. Communication Path is an abstract pattern that can have SDN, TLS, and other communication protocols as concrete subclasses [55]. Classes PE and PA come from the Application Firewall pattern discussed earlier. A more detailed authorization structure is shown in a pattern [56]. The complete SRA requires the integration of the patterns of Fig. 5 with these patterns.

4.4. Threats and governance in ZTA

No system can be considered secure unless a systematic enumeration of threats has been performed; until now, nobody has done a systematic threat enumeration of the threats to the components of a ZTA. We have only found two partial studies. NIST [25] found seven types of threats. Ferreti, at. el [57] also found seven types of threats, but somewhat different from the ones in [25]. Combining similar threats, we can summarize their findings as:

- Subversion of the decision process. Can happen from attacks to PE or PA (control plane). Can result in disruption and unauthorized resource access.
- Attacks to Infrastructure Certification Authority: the attacker can introduce fake components, e.g., a PE or a PA, resulting in unauthorized resource access.
- Denial of service. Blocking access to the interfaces that receive requests can paralyze the system.
- Unauthorized access to security information such as data policies, identity information, trust repositories, device inventories, and similar, can result in unauthorized accesses, denial of service, or just disruption.
- Coercion of automated functions, e.g., trust calculations, can result on disruption or unauthorized accesses.
- Device Certification Authority: compromise of the DCA signing key. Only works if the attacker can also have a legitimate device.

However, this is not a systematic enumeration of threats, which would require the consideration of all the use cases of these components; their analysis would show the goals of the attacker with respect to the assets created or accessed in each use case [7]. Since the use cases define all the interactions with the system, this threat enumeration is systematic and results in a set of threats significantly more complete than ad hoc enumerations. Of course, we still need to enumerate application threats. We can describe threats and their corresponding misuses with appropriate patterns [7].

As indicated, ZTA is a strategy, not a specific architecture, and it also encompasses people and physical aspects. This means that governance aspects are very important; they include assignment of rights to roles, role definition, role and rights management, definition of attributes, etc. [40,58,59,60,61,62]. UCON describes some of these aspects but needs to be complemented with role engineering. Compliance is a related objective, it requires to make sure that the assignment of rights follows the constraints of health, financial, and government regulations [63]. The need to satisfy regulations is another reason to emphasize system development based on applications, not in infrastructure. ZT has been criticized for lack of alignment with risk management and existing governance frameworks [61], so more work on this direction is needed.

5. Evaluating ZTA

There is no data to obtain quantitative measures of ZTAs, our answers are based only in logic and our experience on secure systems and patterns. When considering a security architecture there is the risk of overkill, incompleteness, or excess overhead. Five valid questions for ZTA are:

- Q1. Does ZTA apply many unnecessary checks?
- Q2. Does ZTA introduce an intolerable overhead?
- Q3. Is ZTA implementable for most institutions?
- Q4. Can ZTA control more, fewer, or similar number of threats as compared with other approaches?
- Q5. Does ZT produce a huge number of policies?

Does ZTA apply many unnecessary checks? Secure systems do not need to protect every data item or resource. Some information is intended to be accessible to everybody, e.g., product descriptions; other items cannot be reached from outside the network; some access types are open, e.g., writing in specific places may be open to everybody, e.g., restaurant evaluations. According to the submarine principle of Section 2.1, in some cases we can afford to lose some segments of a system as far as the main system functions are not compromised. This means that protecting everything equally may be an overkill. While this is not a security problem, unnecessary checks take time and increase performance overhead. It is impossible to precisely evaluate the effect of unnecessary checks because there is no statistical data about rights that do not need to be checked; however, it is clear that a system with finer access granularity should be somewhat slower. The specific overhead is very dependent on the type of application.

Does ZTA introduce an intolerable overhead? Enforcing RBAC or even ABAC should incur a reasonable overhead many systems use those models now. Even with the unnecessary checks of question 1, the overhead is probably acceptable. However, adding dynamic access control may be a problem. Even worse, if we need to evaluate trust for each request, the situation is not clear. The calculation of the value of trust is not precisely defined in the architectures that use it to decide access; for example, the trust algorithm in NIST (described in Section 3) should add a significant time in each access evaluation. If a subject earns trust separately for each of its requests, requesting the same resources at different times should require verification and re-evaluation of trust (true only for those systems that use dynamic access control, such as BeyondCorp [23]). In the NIST model no re-evaluation of trust, reauthentication, or reauthorization is needed within a session; continual monitoring with possible reauthentication and reauthorization occurs throughout user transactions. Ref. [26] also uses trust but provides no details of its evaluation. Trust is an area where more work is needed, especially an analysis of its effect on access decision times. Until now, nobody has published overhead figures for a working ZTA.

Is ZTA implementable for most institutions? Institutions will require much help from vendors, but ZTA looks feasible. As indicated in [25], ZT is not a single architecture but "a set of guiding principles for workflow, system design and operations that can be used to improve the security posture of any classification or sensitivity level". A gradual implementation is possible following a careful long-term business--driven planning; maturity models can help the evolution towards ZT [40,60]. The problem with gradual implementations is that the intermediate stages may not be as secure as needed. Here, a systematic secure application methodology can be used to guide the evolution of a system. We believe that patterns provide a systematic and simple roadmap to reach the objectives of ZT.

Can ZTA control more, fewer, or similar number of threats as compared with other approaches? Q4 is the hardest to answer. Quoting Neumann [4], we should look "how to attain demonstrably

trustworthy systems and networks that need to operate under stringent requirements for security, reliability, survivability, and other critical attributes. In particular, we pursue sound foundations for the creation of trustworthy systems and networks that can be easily composed out of subsystems and components, with predictably high confidence that they meet their requirements and also do something sensible when forced to operate outside of the expected normal range of operational conditions". ZT objectives are more modest, they do not talk of reliability or survivability, but they do emphasize trust. Trustworthy systems must protect all the architectural levels, but there is no methodology in ZTA to make sure that all the system levels are covered. ZTA concentrates on controlling the attack surface of the system, but there are threats that are due to application interactions or that affect the security information that may not be controlled by this approach. Virtual or physical perimeter controls have similar weaknesses. Controlling everything through access points misses important attacks. Assume an application requires user inputs, e.g., sales applications. The buyers may or may not to be authenticated. This type of applications also allows CSRF or XSS attacks. However, ZTA should make the work of external attackers much harder, although this is not the case for insiders. As indicated earlier, considering application development and all the architectural levels looks like a better approach. If the system has a Database Management System (DBMS) as is frequently the case, we may need to enforce content-based access control [33], implementations based on microsegments cannot perform this type of access control. The CISA maturity model [60] indicates only recommendations for application security testing throughout the development and deployment process, with regular automated testing of deployed applications, but that is not enough.

Q5. **Does ZT produce a huge number of policies?** This point has been brought up by Bertino [64], who indicates that ZTA makes security management much more complex than already is. When microsegments are used, the number of microsegments can also be enormous. This point is related to Q1 and Q2.

Most proposals rely strongly on microsegmentation, but as mentioned above, that approach has serious flaws [65]: Segmentation allows only separating data and resources by sensitivity level; that is, this is a type of multilevel model [33], which means RBAC and ABAC models need to be added in each layer. Multilevel models are not considered useful in industry because users and data cannot be easily assigned to hierarchic levels. It would be possible to contain in one segment all the resources accessible to a specific role, but then they would all be fully accessible; that is, we cannot separate access types. If we divide the sensitivity levels or the roles into many segments, we need many gateways which adds complexity and overhead. In summary, microsegmentation cannot enforce fine security controls and we do not consider it a good way to obtain zero trust. A more logical and precise approach enforces access control with respect to applications, where the semantics of the data defines who is given what type of access. Secure application development allows to use only the needed system defenses and control their interactions. Each application must be developed following a systematic secure development methodology, that starts from enumerating its threats and adds countermeasures to stop them [3]. Otherwise, there is no basis to assign proper rights to users; if this assignment has no functional basis, checking every access will not stop all attacks. To build secure systems we need a global approach, securing specific parts of the system will not result in a secure system; disjoint mechanisms cannot prevent threats that are combinations of authorized actions. Security should be based on institution policies, which in turn depend on the type of application and must be enforced through applications. Methods based on implementation-oriented approaches, e.g., networks or cryptography, are very important to enforce the high-level restrictions, but can never provide precise security.

6. Directions for research

Q1 to Q5 above are effectively directions for research, but more urgent than the ones on this list. From our study we identified several areas that could be interesting research topics:

—Survey the commercial offerings of ZT and compare them in a table. We looked at some of them in Section 3 but there are several others.
—The NIST model and some commercial systems use trust as a factor in deciding access. Define trust precisely, indicate how to apply when deciding access, and analyze its overhead. A good analysis can be found in [66].
—The inputs to the NIST model include aspects such as SIEM, compliance, and data access policies. Study how to efficiently use those inputs in access decisions.
—Build a Security Reference Architecture for ZT. A partial model is shown here, but more details are needed.
—Build a prototype for a ZTA and measure its security overhead and degree of security. This should be a realistic system, not a toy system.
—Write patterns for SDP and SDN.
—The patterns mentioned in Section 4.3 are abstract patterns, not tied to any specific context; it is necessary to extend them to match the context of specific types of applications. This would allow to build application-oriented ZT RAs.
—Study the combination of SDN and SDP to implement ZT.
—Databases are protected using a combination of authentication, authorization, and cryptography. Show how these defenses can be used in a ZT environment.
— Performance studies of the use of cryptography in ZT networks are needed. Propose intelligent ways to use cryptography, not brute force approaches.
—Do a systematic enumeration of the threats to the security components in a ZTA. We showed a partial threat enumeration in the paper, but we need a systematic enumeration using the method in [7].
—Develop governance models for ZTAs. Consider UCON, role engineering, and alignment with governance frameworks [61].
—Collect data from real systems to try to answer more precisely Q1 to Q5.
—Answer Q4 by finding threats that cannot be handled by a pure ZTA. An example is given in Section 5.
—Combine a secure systems development methodology such as the ones mentioned in [3], with a ZT approach.

7. Related work

We examine now two types of work: overviews of the subject and studies of specific topics. Note that this is not a systematic survey, we only mention works that refer to the topics discussed in our analysis.

There are only a few works that provide a general overview of ZTA:

Buck et al. [67], published a multivocal literature survey of ZTA, including technical aspects and management and economic aspects, which we do not consider here.

Ferreti et al. [57] provided a brief survey of ZTAs, considered a network-driven ZTA, and secured the control plane by analyzing its threats and providing defenses. They also showed a basic reference architecture for ZT, and proposed a ZTA that can recover from failures and attacks. This, and [25], seem to be the only works discussing threats to ZTA.

Ref. [68] provided an overview of ZTA concepts; in particular of the NIST model.

Ref. [69] performed a survey of key technologies of zero trust showing some environments to highlight the advantages of zero trust (Big data, Clouds, IoT), and ideas for future research.

Five reports summarized ZTA concepts and development, intended

for its adoption by the US Government: [70] surveyed the state of the art until 2020, NIST [25] summarized work on ZTA until early 2020. Ref. [40] considered mobility environments, while [60] discussed maturity models. The Department of Defense of the USA proposed a RA oriented to military applications [71]. None of these works is a critical analysis, they mostly describe different aspects of the ZTA.

Syed et al. [72] presented a comprehensive survey with many references but little on systems security aspects, no models or directions for research.

There is also already a considerable amount of research discussing specific aspects, we discuss here only a few; for a longer list and a systematic survey see [67] and [72].

Yao et al. [73] proposed a dynamic access control and authorization system for ZTA. The authors proposed a trust-based access control (TBAC) which is based on the calculation of a behavioral trust (BT) score for each user. In the case that BT exceeds the minimum trust score required to access the resource, access permission is granted.

Vanickis et al. [74] attempted to adapt the Context Handler entity and the UCON model for zero trust as a risk policy enforcement framework added on the network layer. However, instead of XACML, that research proposed using their own language, the PAROLE policy language.

Performance aspects of Software Defined Perimeter (SDP) is the focus of [75], and [76] described pros and cons of SDP, compared with traditional network access models.

As a common trend (also, as a way to get a bit of hype), blockchain has been considered regarding zero trust [77]. Samaniego and Deters [78] presented a blockchain-based middleware of decentralized access control for the IoT domain, based on zero trust principles. And Zhao et al. [79] proposed blockchain-based user authentication in zero trust IoT.

Brancik and Bertino discussed services for ZTAs [80] and presented ideas for implementation. However, they considered the use of threat catalogs like CVE, that include very large sets of threats instead of threats based on goals that are a much smaller set. They indicate that security patterns are critical to apply controls in ZTA. In a subsequent paper [81], that group proposed a graph-based approach to define all the needed network policies of the system; they assumed micro-segmentation as enforcement mechanism.

The application of ZT to the design of the complex architectures present in 5 G/6 G networks has been studied by Ramenzapour and Jagannath [82].

8. Conclusions and future work

We have analyzed several aspects of ZTA, including architectural and implementation aspects using the patterns involved in this architecture as a way to relate the architecture to classical security theory. This connection was used to answer some basic questions about ZTA. We found that in current work there are several issues that are not discussed or superficially discussed such as performance, threats, and governance. There are very few reports on the actual use of ZTA in practice. There is no published experience with implementations, showing actual measurements of performance, estimates of their degree of security, ease of use, or other quality factors. Without them, it is hard to know what to expect before starting a ZTA project. Although there are a few reports of actual implementations, they are only descriptive, and do not mention any quality measures. All this led to the definition of a set of directions for research. As far as we know, nobody has analyzed ZT from a point of view of classical security theory, decomposed ZTAs into patterns, and proposed a reference architecture for ZT based on patterns.

Our future work is focused on the completion of the preliminary Security Reference Architecture of Section 4.3; this work will require to write some new security patterns, adapt some existing security patterns and combining them into a RA, to be followed by a systematic enumeration of threats and finding appropriate defenses to build a SRA.

CRediT authorship contribution statement

Eduardo B. Fernandez: Conceptualization, Investigation, Methodology, Writing – original draft, Writing – review & editing. **Andrei Brazhuk:** Investigation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgements

We thank the referees for their comments that significantly improved this paper.

Appendix: Glossary

ABAC: Attribute-Based Access Control
BYOD: Bring Your Own Device
CASB: Cloud Access Security Broker
CDM: Continuous Diagnostics and Mitigation
CISA: Cybersecurity and Infrastructure Security Agency
CTI: Cyber Threat Intelligence
DAN: Data Acquisition Network
DCA: Device Certification Authority
LTE: Long Term Evolution
MCAP: Microcore and Perimeter mTLS: mutual Transport Layer Security
NCCoE: National Cybersecurity Center of Excellence
NGFW: Next Generation Firewall
NIST: National Institute of Standards and Technology
PA: Policy Administrator
PAP: Policy Administration Point or Policy Authorization Point
PDP: Policy Decision Point
PE: Policy Engine
PEP: Policy Enforcement Point
PIP: Policy Information Point
PKI: Public Key Infrastructure
RA: Reference Architecture
RBAC: Role-Based Access Control
SASE: Secure Access Service Edge
SDN: Software Defined Network
SDP: Software Defined Perimeter
SG: Secure Gateway
SIEM: Security Information and Event Management
SPA: Single-Packet Authorization
SRA: Security Reference Architecture
SSO: Single Sign-On
TBAC: Trust-based Access Control
UCON: Usage Control
UML: Unified Modeling Language
UTM: Unified Threat Management
VLAN: Virtual Local Area Network
WFH: Work From Home
XACML: eXtensible Access Control Markup Language
ZT: Zero Trust
ZTA: Zero Trust Architecture
ZTNA: Zero Trust Network Access
ZTX: Zero Trust Extended

References

[1] J.H. Saltzer, M.D. Schroeder, The protection of information in computer systems, Proc. IEEE 63 (9) (1975).
[2] J. Shapiro, EROS: a principle-driven operating system from the ground up, IEEE Softw. 19 (2002) 26–33. January/February.
[3] A. Uzunov, E.B. Fernandez, K. Falkbeer, Assessing and improving the quality of security methodologies for distributed systems, J. Softw. 30 (11) (2018), https://doi.org/10.1002/smr.1980.
[4] P.G. Neumann, Achieving principled assuredly trustworthy composable systems and networks, in: Proceedings of the DARPA Information Survivability Conference and Exposition, 2003, https://doi.org/10.1109/DISCEX.2003.1194962, 22-24 April.
[5] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, Stal. M. Pattern-Oriented Software Architecture: A System of Patterns, 1, J. Wiley, 1996.
[6] E. Gamma, R. Helm, R. Johnson, J. Vlissides, Design Patterns –Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.
[7] E.B. Fernandez, Security Patterns in Practice: Designing Secure Architectures Using Software Patterns, Wiley, 2013.
[8] Wikipedia: equifax data breach. Available online https://en.wikipedia.org/wiki/2017_Equifax_data_breach (accessed on Feb.5, 2022).
[9] Forbes, Massive Security Breach At Sony – Here's What You Need To Know, 2014. https://www.forbes.com/sites/josephsteinberg/2014/12/11/massive-security-breach-at-sony-heres-what-you-need-to-know/?sh=6bad280144d8 (accessed on Feb.5, 2022).
[10] E.B. Fernandez, N. Yoshioka, H. Washizaki, M. VanHilst, An approach to model-based development of secure and reliable systems, in: Proceedings of the Sixth International Conference on Availability, Reliability and Security (ARES 2011), Vienna, Austria, 2011. August 22-26.
[11] P.G. Neumann, On hierarchical design of computer systems for critical applications, IEEE Trans. Software Eng. 12 (9) (1986) 905–920.
[12] O. Gadyatskaya, F. Massacci, Y. Zhauniarovich, Security in the Firefox OS and Tizen mobile platforms, Comput. IEEE 47 (6) (2014) 57–63, https://doi.org/10.1109/MC.2014.165.
[13] E.B. Fernandez, S. Mujica, F. Valenzuela, Two security patterns: least privilege and security logger/auditor, Procs. Asian PLoP (2011).
[14] E.B. Fernandez, H. Washizaki, N. Yoshioka, T. Okubo, The design of secure IoT applications using patterns: state of the art and directions for research, Internet of Things 15 (2021).
[15] H. Washizaki, et al., Software engineering patterns for machine learning applications (SEP4MLA), in: Part 2 Proceedings of the 27th Conference on Pattern Languages of Programs, 2020, pp. 1–10.
[16] R.N. Taylor, N. Medvidovic, N. Dashofy, Software architecture: Foundation, theory, and Practice, Wiley, 2010.
[17] A. Brazhuk, Threat modeling of cloud systems with ontological security pattern catalog, Int. J. Open Info. Technol. 9 (5) (2021) 36–41.
[18] E.B. Fernandez, R. Monge, K. Hashizume, Building a security reference architecture for cloud systems, Requirements Eng. 21 (2) (2016) 225–249.
[19] J. Kindervag, Build security into your network's dna: the zero trust network architecture, Forrester Res. Inc. (2010) 1–26.
[20] Maddison, J. Working Securely from Anywhere With Zero Trust. 2021. https://www.securityweek.com/working-securely-anywhere-zero-trust (accessed on 2 Feb 2022).
[21] Riley, S.; MacDonald, N. Market Guide for ZTNA. 8 June 2020. https://www.gartner.com/doc/reprints?id=1-25BCTFZJ&ct=210224&st=sb (accessed on Feb.2, 2022).
[22] Cunningham, C. The Zero Trust eXtended (ZTX) Ecosystem Extending Zero Trust Security Across Your Digital Business. 2018. https://www.cisco.com/c/dam/m/en_sg/solutions/security/pdfs/forrester-ztx.pdf (accessed on Feb.2, 2022).
[23] Ward, R., Beyer, B. BeyondCorp: a New Approach to Enterprise Security.;login:, December 2014, Vol. 39, No. 6, 6–11.
[24] Akamai Corp, How-To Guide: zero Trust Security Transformation. https://www.akamai.com/resources/white-paper/how-to-guide-zero-trust-security-transformation (accessed on Feb.2, 2022).
[25] S. Rose, O. Borchert, S. Mitchell, S. Connelly, Zero Trust Architecture, Special Publication (NIST SP-800-207), 2020, https://doi.org/10.6028/NIST.SP.800-207. August.
[26] Qi Anxin, Zero Trust Architecture and solutions, Apr 26, 2020. https://www.gartner.com/teamsiteanalytics/servePDF?g=/imagesrv/media-products/pdf/Qi-An-Xin/Qi-An-Xin-1-1OKONUN2.pdf (accessed on Feb.2, 2022).
[27] Implementing a Zero Trust security model at Microsoft, March 19, 2021. https://www.microsoft.com/en-us/insidetrack/implementing-a-zero-trust-security-model-at-microsoft (accessed on Feb.2, 2022).
[28] T. Lukaseder, M. Halter, F. Kargl, Context-based Access Control and Trust Scores in Zero Trust Campus Networks, SICHERHEIT, 2020.
[29] Z. Zaheer, H. Chang, S. Mukherjee, J. Van der Merwe, Eztrust: network-independent zero-trust perimeterization for microservices, in: Proceedings of the 2019 ACM Symposium on SDN Research, 2019, pp. 49–61. April.
[30] NCCoE, Implementing a Zero Trust Architecture. NIST Special publication 1800-35D https://www.nccoe.nist.gov/projects/implementing-zero-trust-architecture (accessed on October 30, 2023).
[31] Cloudfare, A roadmap to zero trust architecture, available from cloudfare.com (accessed on October 30, 2023).
[32] OASIS eXtensible Access Control Markup Language (XACML) Technical Committee. https://www.oasis-open.org/committees/tc_home.php?wg_abbrev=xacml (accessed on Feb.2, 2022).
[33] Gollmann, D., Computer Security (3rd Ed.), Wiley, 2011.
[34] T. Priebe, E.B. Fernandez, J.I. Mehlau, G. Pernul, A Pattern System for Access Control, in Research Directions in Data and Applications Security XVIII, in: C. Farkas, P. Samarati (Eds.), Proceedings of the 18th. Annual IFIP WG 11.3 Working Conference on Data and Applications Security, Sitges, Spain, 2004, pp. 235–249. July 25–28.
[35] J. Park, R. Sandhu, The UCON usage control model, ACM Secur. TISSEC 7 (1) (2004) 128–174.
[36] N. Delessy, E.B. Fernandez, S. Rajput, M.M. Larrondo-Petrie, Patterns for application firewalls, in: Proceedings of the Pattern Languages of Programs (PLoP) Conference, 2004.
[37] N. Delessy, E.B. Fernandez, Patterns for the eXtensible access control markup, in: Proceedings of the Pattern Languages of Programs (PLoP) Conference, 2005.
[38] Cser, A. XACML is dead, May 7, 2013. https://www.forrester.com/blogs/13-05-07-xacml_is_dead/(accessed on Feb.2, 2022).
[39] Delessy, N.; Fernandez, E.B.; Larrondo-Petrie, M.M. A pattern language for identity management. Procs. of the 2nd IEEE Int. Multiconference on Computing in the Global Information Technology (ICCGI 2007), March 4-9, Guadeloupe, French Caribbean.
[40] Cybersecurity and Infrastructure Security Agency, Applying Zero Trust Principles to Enterprise Mobility, March 2022. https://www.cisa.gov/sites/default/files/publications/Zero_Trust_Principles_Enterprise_Mobility_For_Public_Comment_508C.pdf (accessed on Feb. 2, 2022).
[41] E.B. Fernandez, N. Yoshioka, H. Washizaki, Abstract and IoT security segmentation patterns, in: Proceedings of the 8th Asian Conf. Pattern Lang. Programs (AsianPLoP), 2019, pp. 1–9.
[42] W.R. Simpson, K.E. Foltz, Network segmentation and zero trust architectures, Proc. WCE 2021 (2021). July 7-9.
[43] VMware, VMware Network and Micro-Segmentation. https://www.vmware.com/content/dam/digitalmarketing/vmware/en/pdf/solutions/vmware-network-micro segmentation-solution-overview.pdf (accessed on Feb.2, 2022).
[44] A. Osman, A. Wasicek, S. Köpsell, T. Strufe, Transparent microsegmentation in smart home IoT networks, in: Proceedings of the 3rd {USENIX} Workshop on Hot Topics in Edge Computing (HotEdge 20), 2020.
[45] C. DeCusatis, P. Liengtiraphan, A. Sager, M. Pinelli, Implementing zero trust cloud networks with transport access control and first packet authentication, in: Proceedings of the 2016 IEEE International Conference on Smart Cloud (SmartCloud), IEEE, 2016, pp. 5–10. November.
[46] A. Moubayed, A. Refaey, A. Shami, Software-defined perimeter (sdp): state of the art secure solution for modern networks, .IEEE Netw. 33 (5) (2019) 226–233.
[47] M. Shlapentokh-Rothman, E. Hemberg, U.M. O'Reilly, Securing the software defined perimeter with evolutionary co-optimization, in: Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion, 2020, pp. 1528–1536.
[48] X. Xie, G. Gan, Y. Chen, Research on SDP software defined perimeter initiating host protocol configuration algorithm, in: Proceedings of the IOP Conference Series: Earth and Environmental Science 428, 2020.
[49] D. Kreutz, F.M. Ramos, P.E. Verissimo, C.E. Rothenberg, S. Azodolmolky, S. Uhlig, Software-defined networking: a comprehensive survey, Proc. IEEE 103 (1) (2014) 14–76.
[50] A. Shaghaghi, M.A. Kaafar, R. Buyya, S. Jha, Software-defined network (SDN) data plane security: issues, solutions, and future directions, Handbook Comput. Netw. Cyber Secur. (2020) 341–387.
[51] R. Sahay, G. Blanc, Z. Zhang, K. Toumi, H. Debar, Adaptive policy-driven attack mitigation in SDN, in: Proceedings of the 1st International Workshop on Security and Dependability of Multi-Domain Infrastructures, 2017, pp. 1–6. April.
[52] A. Sallam, A. Refaey, A. Shami, On the security of SDN: a completed secure and scalable framework using the software-defined perimeter, IEEE Access 7 (2019) 146577–146587.
[53] Uzunov, A., Fernandez, E.B. An Extensible Pattern-based Library and Taxonomy of Security Threats for Distributed Systems. Special Issue on Security in Information Systems of the Journal of Computer Standards & Interfaces. 2013. 10.1016/j.csi.2013.12.008.
[54] E.B. Fernandez, N. Yoshioka, H. Washizaki, Cloud Access Security Broker (CASB): a pattern for accessing secure cloud services, in: Proceedings of the 4th AsianPLoP (Pattern Languages of Programs) 2015, Tokyo, Japan, 2015. March.
[55] A.Brazhuk E.B.Fernandez, The abstract secure communication pattern (ASCP) and a derived VPN pattern, in: Proceedings of the Latin American Int. Conf. on Pattern Languages of Programs, 2022. October.
[56] A. Brazhuk, E.B. Fernandez, An abstract security pattern for Zero Trust access control, in: Proceedings of the 29th Int. Conf. on Pattern Languages of Programs, 2022. October.
[57] L. Ferretti, F. Magnanini, M. Andreolini, M. Colajanni, Survivable zero trust for cloud computing environments, Comput. Secur. 110 (2021), https://doi.org/10.1016/j.cose.2021.102419.
[58] L. Fuchs, G. Pernul, R. Sandhu, Roles in information security–a survey and classification of the research area, Comput. Secur. 30 (8) (2011) 748–769.
[59] R.V. Solms, B.V. Solms, Information security governance; a model based on the direct-control cycle, Comput. Secur. 25 (6) (2006) 408–412, 17.
[60] Cybersecurity and Infrastructure Security Agency Cybersecurity Division, Zero Trust Maturity Model, Pre-decisional Draft June 2021 Version 1. https://www.cisa.gov/sites/default/files/publications/CISA Zero Trust Maturity Model_Draft.pdf (accessed on Feb.2, 2022).
[61] Bobbert Yuri, Scheerder Jeroen, Zero trust validation: from practical approaches to theory, Sci. J. Res. Reviews (2022), https://doi.org/10.33552/SJRR.2020.02.000546 (accessed on March 25.

[62] L. Fuchs, G. Pernul, Prorole: a process-oriented lifecycle model for role systems leveraging identity management and guiding role projects, ECIS (2008) 1322–1333.
[63] Yimam, D.; Fernandez, E.B. A Survey of compliance issues in cloud computing, Journal of Internet Services and Applications, 2016, 7:5, 10.1186/s13174-016-0046-8, http://www.jisajournal.com/content/7/1/5.
[64] E. Bertino, Zero Trust Architecture: does It Help? IEEE Secur. Priv. 19 (5) (2021) 95–96.
[65] Haber, M.J. Why Zero Trust is an unrealistic security model", Beyond Trust. Available online: https://www.beyondtrust.com/blog/entry/why-zero-trust-is-an-unrealistic-security-model (accessed on Feb.2, 2022).
[66] N.Gol Mohammadi, Trustworthy Cyber-Physical Systems, Springer Vieweg, 2018.
[67] C. Buck, C. Olenberger, A. Schweizerc, F. Völter, T. Eymanne, Never trust, always verify: a multivocal literature review on current knowledge and research gaps of zero-trust, Comput. Secur. 110 (2021), https://doi.org/10.1016/j.cose.2021.10243.
[68] S. Teerakanok, T. Uehara, A. Inomata, Migrating to zero trust architecture: reviews and challenges, Secur. Commun. Netw. (2021).
[69] Yan, X.; Wang, H., Survey on zero-trust network security. Communications in Computer and Information Science. Artificial Intelligence and Security, 2020; Vol. 1252. Singapore: Springer; pp. 50–60. doi:10.1007/978-981-15-8083-3_5.
[70] K.D. Uttecht, Zero Trust concepts For Federal Government Architectures, MIT Lexington, 2020. Tech. Report.
[71] Department of Defense (DoD). (2022). Zero Trust Reference Architecture. https://dodcio.defense.gov/Portals/0/Documents/Library/(U)ZT_RA_v2.0(U)_Sep22.pdf.
[72] N.F. Syed, A. Shaghaghi, Z. Baig, R. Doss, Zero Trust Architecture (ZTA): a comprehensive survey, IEEE Access 10 (2022) 57143–57179, https://doi.org/10.1109/ACCESS.2022.3174679.
[73] Q. Yao, Q. Wang, X. Zhang, J. Fei, Dynamic access control and authorization system based on zero-trust architecture, in: Proceedings of the 2020 International Conference on Control, Robotics and Intelligent System, Xiamen, China, 2020. October.
[74] R. Vanickis, P. Jacob, S. Dehghanzadeh, B. Lee, Access control policy enforcement for zero-trust-networking, in: Proceedings of the 29th Irish Signals and Systems Conference (ISSC), IEEE, 2018, pp. 1–6. June.
[75] P. Kumar, A. Moubayed, A. Refaey, A. Shami, J. Koilpillai, Performance analysis of SDP for secure internal enterprises, in: Proceedings of the 2019 IEEE Wireless Communications and Networking Conference (WCNC), IEEE, 2019, pp. 1–6.
[76] R.R. Omar, T.M. Abdelaziz, A comparative study of network access control and software-defined perimeter, in: Proceedings of the 6th International Conference on Engineering & MIS 2020, 2020, pp. 1–5.
[77] Alevizos, L.; Ta, V.T.; Eiza, M.H. Augmenting Zero Trust Architecture to Endpoints Using Blockchain: a Systematic Review. 2021; arXiv preprint.
[78] Samaniego, M.; Deters, R.; Zero-trust hierarchical management in IoT. In Proceedings of the 2018 IEEE international congress on Internet of Things (ICIOT, 2018); pp. 88–95. IEEE.
[79] S. Zhao, S. Li, F. Li, W. Zhang, M. Iqbal, Blockchain-Enabled user authentication in zero trust internet of things, in: Proceedings of the Int. Conf. on Security and Privacy in New Computing Environments, Springer, 2020, pp. 265–274.
[80] E. Bertino, K. Brancik, Services for zero trust architectures - a research roadmap, in: Proceedings of the 2021 IEEE International Conference on Web Services (ICWS), 2021, pp. 14–20, https://doi.org/10.1109/ICWS53863.2021.00016 (accessed on April 4, 2022).
[81] C. Katsis, F. Cicala, D. Thomsen, N. Ringo, E. Bertino, NEUTRON: a graph-based pipeline for zero-trust network architectures, in: Proceedings of the Twelfth ACM Conference on Data and Application Security and Privacy, 2022, pp. 167–178 (accessed on May 2nd 2022).
[82] K. Ramezanpour, J. Jagannath, Intelligent zero trust architecture for 5G/6G networks: principles, challenges, and the role of machine learning in the context of O-RAN, Comput. Netw. 217 (2022) 109358, https://doi.org/10.1016/J.COMNET.2022.109358. Nov.